Super Hacker Level 1 – Python Report

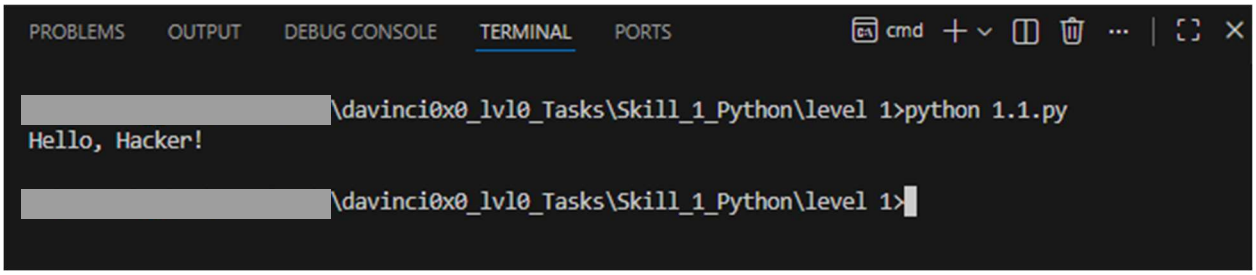
Level 1: Python Basics & Data Types

Files path : davinci0x01_lvl0_Tasks\Skill_1_Python\level 1

1. Print "Hello, Hacker!" to the console > 1.1.py

- I use print() in python to print this string

Screenshot:



2. Ask the user for their name and print "Welcome, [name]!" > 1.2.py

- I use variable to store user input and use input() to make user able to enter his name and after that uses print() with f before quotes to define variable

Screenshot:

3. Convert "1337" (string) to an integer and add 10 > 1.3.py

- First make variable to store str value

- use print() and in it built-in functions that convert str to int called int() and sum 10

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   cmd

davinci0x0_lvl0_Tasks\Skill_1_Python\level 1>python 1.3.py
1347

davinci0x0_lvl0_Tasks\Skill_1_Python\level 1>
```

4. Create a program that asks for two numbers and prints their sum, difference, ,product and quotient > 1.4.py

- First make two variables to store and take user input num1 and num2

- use + for sum , - for difference , * for product and / for quotient

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   cmd

davinci0x0_lvl0_Tasks\Skill_1_Python\level 1>python 1.4.py
Enter first number: 4
Enter second number: 2
The sum is: 6
The product is: 8
The difference is: 2
The quotient is: 2.0

davinci0x0_lvl0_Tasks\Skill_1_Python\level 1>
```

5. Reverse the string "rekcah_repus" without using built-in functions > 1.5.py

- I make function that called reverse() take variable called string that use python slicing to re-create the string reversed using string[: :-1]

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   cmd

[redacted]\davinci0x0_lvl0_Tasks\Skill_1_Python\level 1>python 1.5.py
super_hacker

[redacted]\davinci0x0_lvl0_Tasks\Skill_1_Python\level 1>
```

6. Write a program that checks if a number is even or odd > 1.6.py

- Odd numbers are numbers that are not evenly divisible by 2

- I make function that check if num can be divided by 2 or not using % in python that check if number has remainder of division

- So if number is even the % return Zero

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   cmd

[redacted]\davinci0x0_lvl0_Tasks\Skill_1_Python\level 1>python 1.6.py
Enter a number to check: 4
the number is even

[redacted]\davinci0x0_lvl0_Tasks\Skill_1_Python\level 1>python 1.6.py
Enter a number to check: 3
the number is odd

[redacted]\davinci0x0_lvl0_Tasks\Skill_1_Python\level 1>
```

7. Create a variable that stores a boolean value and print it > 1.7.py

- Make variable and store True on it or you can store False They are the same thing

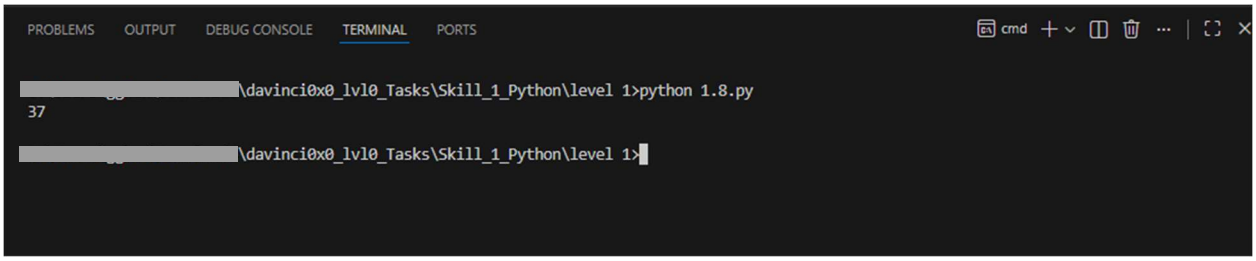
- Use print() to print value

Screenshot:

8. Convert the string "100101" to a decimal number > 1.8.py

- I use int() with Radix 2 for binary number

Screenshot:

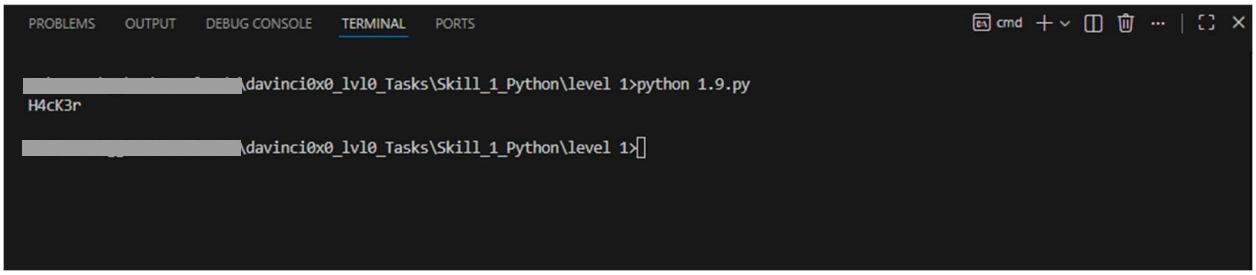


9. Print "H4ck3r" with alternating uppercase and lowercase letters > 1.9.py

- I make 3 variables phrase to store the string, status with int value to make the program alternating, valid_phrase to store final result

- I create for loop to check every char in phrase and make the frist one uppercase and change value of status to make the next char lowercase

Screenshot:



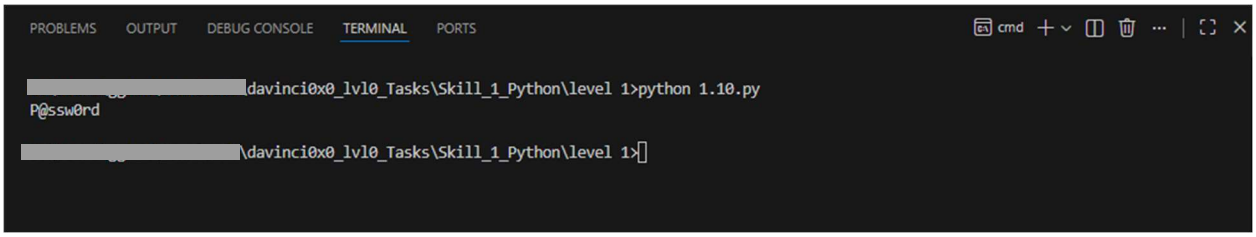
10. Replace all vowels in "P@ssw0rd" with " * " > 1.10.py

- First make variable to store phrase we want to ckeck

- I make another value contain vowels "aeiouAEIOU" and empty string variable to store the filnal result

- Create for loop to check every char in phrase if it's in vowels and add char to the final result variable

Screenshot:



Level 2: Control Flow & Loops

Files path : davinci0x01_lvl0_Tasks\Skill_1_Python\level 2

1. Write a script that asks for a password and only allows access if it matches "s3cr3t" > 2.1.py

- First I make variable to store the password and make variable to take user input and store it

- make validation for user input

Screenshot:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   cmd

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>python 2.1.py
Welcome! Please enter the password to continue.
Enter the password: s3cr3t
Access granted.
############################
Welcome to the secret area!
############################

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>python 2.1.py
Welcome! Please enter the password to continue.
Enter the password: 123
Access denied. Incorrect password.

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>
```

2. Print all numbers from 1 to 100 except numbers divisible by 4 > 2.2.py

- First make for loop in range(1, 101) to make list from 1 to 100

- make validation for every number in list if it not divisible by 4

Screenshot:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   cmd

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>python 2.2.py
1
2
3
5
6
7
9
10
11
13
14
15
```

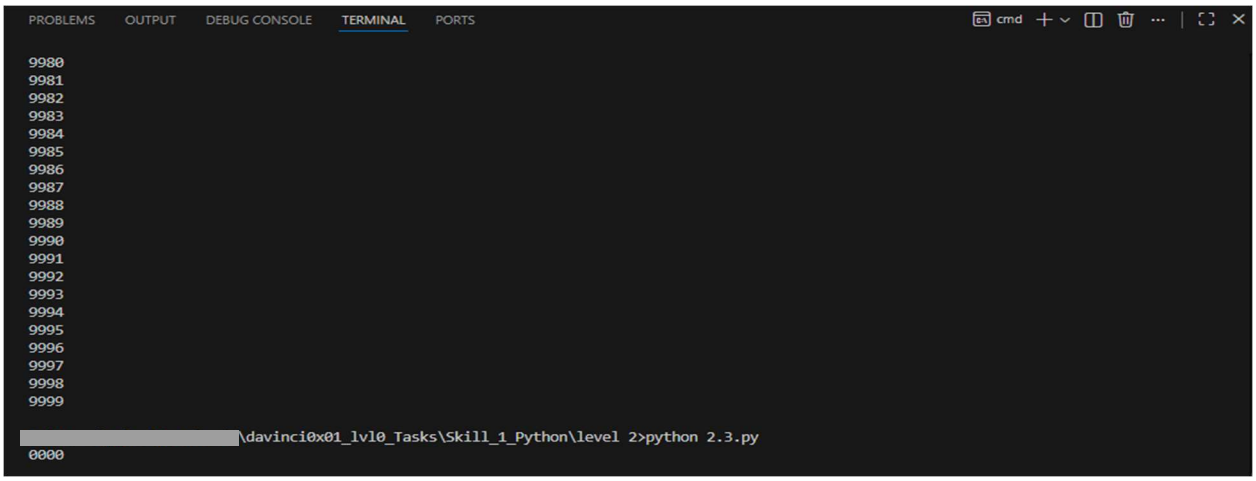
3. Print every 4-digit PIN code (0000 – 9999) > 2.3.py

- First make for loop in range from 0 to 9999

- Use f-string formatting, be like in print() >> f" {value:format}"

- In format section I use 04d makes the number always appear as four digits by adding leading zeros

Screenshot: result with normal code & result with break at first loop

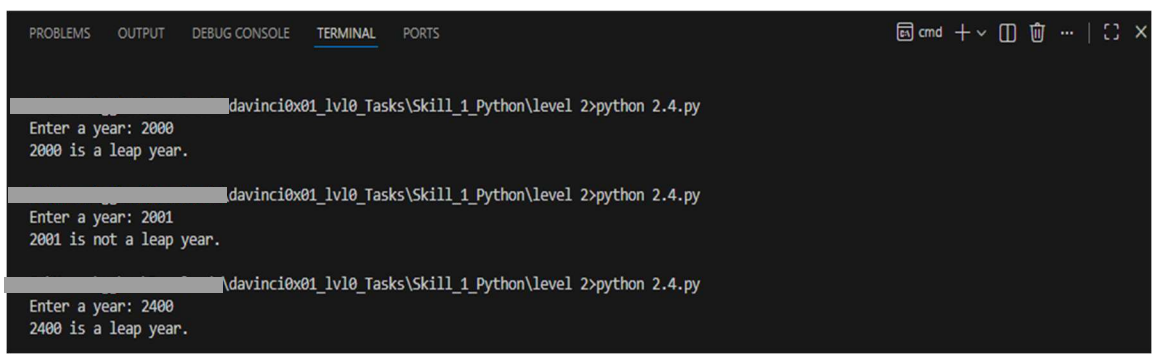


4. Check if a given year is a leap year > 2.4.py

- A leap year is any year that is exactly divisible by 4, not divisible by 100, unless it is also divisible by 400

- Based on these rules, I validated the user input and then checked whether the given year satisfies the leap-year conditions

Screenshot:



5. Find prime numbers between 1 and 100 > 2.5.py

- Prime number that can only be divided by 1 and by itself with no remainder and greater than 1

- So in every time I check if it can be divided by other numbers without remainder

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   cmd

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>python 2.5.py
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
53
59
61
67
71
73
```

6. Simulate a login system that locks after 3 failed attempts > 2.6.py

- Frist I make variable to store password and another one for attempts

- I use While loop to make script always on and repeat input and check it

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>python 2.6.py
Enter the password: davinci
Incorrect password,You have 2 attempts left.
Enter the password: davinci1
Incorrect password,You have 1 attempts left.
Enter the password: davinci12
Incorrect password,You have 0 attempts left.
Access denied. No attempts left.

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>python 2.6.py
Enter the password: davinci0x01
Access granted.
#############################
Welcome Sir
#############################
```

7. Create a number guessing game > 2.7.py

"This game I made it before in cs50 course"

- First we need to import from random library randint function to choose random number every time

- Create two variables to store the random number and chances

- Create infinity loop with while loop and Type game logic with IF condition

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>python 2.7.py
Guess a number between 1 and 25: 10
Too high
Guess a number between 1 and 25: 5
Too high
Guess a number between 1 and 25: 1
Too low
Guess a number between 1 and 25: 3
Too low
Guess a number between 1 and 25: 4
Correct
The number is 4
```

8. Print numbers 1-100, replacing multiples of 3 with "Fizz", 5 with "Buzz", both with "FizzBuzz" > 2.8.py

- First make list from 1 to 100 with range function

- If a number is divisible by 3 or 5 it means it is a multiple of either

Screenshot:

```
Fizz
88
89
FizzBuzz
91
92
Fizz
94
Buzz
Fizz
97
98
Fizz
Buzz
```

9. Continuously ask for a password until the correct one is entered > 2.9.py

- Create infinity while loop with IF condition to ask for password until entered correct

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>python 2.9.py
Enter the password: 123
Enter the password: davinci
Enter the password: davinci123
Enter the password: davinci@123
Enter the password: davinci0x0
Enter the password: davinci0x0123
Enter the password: 123456
Enter the password: davinci0x01
Access granted
```

10. Check if a string is a palindrome > 2.10.py

- To know if string is Palindrome we need to reverse it and check if it the same with the original one

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>python 2.10.py
Enter a string: radar
"radar" is a palindrome

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 2>python 2.10.py
Enter a string: nmap
"nmap" is not a palindrome
```

Level 3: Lists, Tuples, and Dictionaries

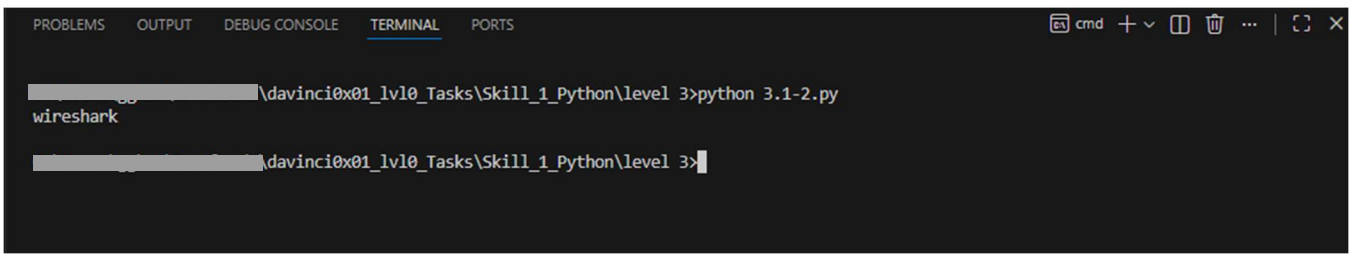
Files path : davinci0x01_lvl0_Tasks\Skill_1_Python\level 3

1. Create a list of 10 hacker tools | 2. Print the 3rd item in the list > 3.1-2.py

- Create variable contain list with tools

- Use list index to print the 3rd item

Screenshot:



3. Create a dictionary of HTTP status codes > 3.3.py

- To create dictionary we use curly brackets { }

Screenshot:

4. Count how many times each letter appears in a string > 3.4.py

- First create variable to store the string I store "Hello, World!"

- Create 2 variables for the letter we want to search for and another for how many times it appears

- Make for loop to check if the letter in string and add how many times show it

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   cmd

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.4.py
The letter 'l' appears 3 times in the string.

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.4.py
The letter 'o' appears 2 times in the string.

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>
```

5. Sort a list of random numbers without using .sort()

- Create variable contain the nums and another one for the sorted nums

- Make while nums loop to run as long as the list is not empty

- When find smallest append it to sorted list and remove it from the main list

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   cmd

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.5.py
[3, 5, 2, 10, 8, 6, 1, 4, 7, 9]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>
```

6. Store ports and services in a dictionary and allow user queries

- Create dictionary of ports

- Make user input and check if it int and search if it in dictionary and print it

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.6.py
Enter a port number: 80
Port 80 uses HTTP service.

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.6.py
Enter a port number: 443
Port 443 uses HTTPS service.

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.6.py
Enter a port number: 222
Port not found in the dictionary.

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.6.py
Enter a port number: fff
Please enter a valid port number.
```

7. Function to remove duplicates from a list

- Create function take list as input and in it empty list to store the result

- Make for loop to check for every item in list if it duplicates

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.7.py
[3, 5, 2, 10, 8, 6, 1, 4, 7, 9, 3, 5, 2]
[3, 5, 2, 10, 8, 6, 1, 4, 7, 9]

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>
```

8. Convert a list into a comma-separated string

- Create list variable contain the strings and another one for result

- Use .join() function to append comma between every item

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.8.py
Nmap, Wireshark, Metasploit, Burp Suite

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>
```

9. Find the longest word in a list

- Create 3 variables: one to contain a list of strings, the second for the result, and the third for the maximum length of a string

- Make a for loop to check every string to see if it is the longest

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   cmd

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.9.py
The longest item is: Metasploit

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>
```

10. Given a dictionary of usernames and passwords, print the password for a given Username

- Create a dictionary that stores usernames as keys and passwords as values

- Ask the user to enter a username

- Check if the username exists in the dictionary

- If it exists, print the password for the username

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   cmd

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.10.py
Enter username: admin
Password: admin123

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 3>python 3.10.py
Enter username: davinci
Username not found.
```

Level 4: Functions & Modules

Files path : davinci0x01_lvl0_Tasks\Skill_1_Python\level 4

1. Write a function that reverses a string > 4.1.py

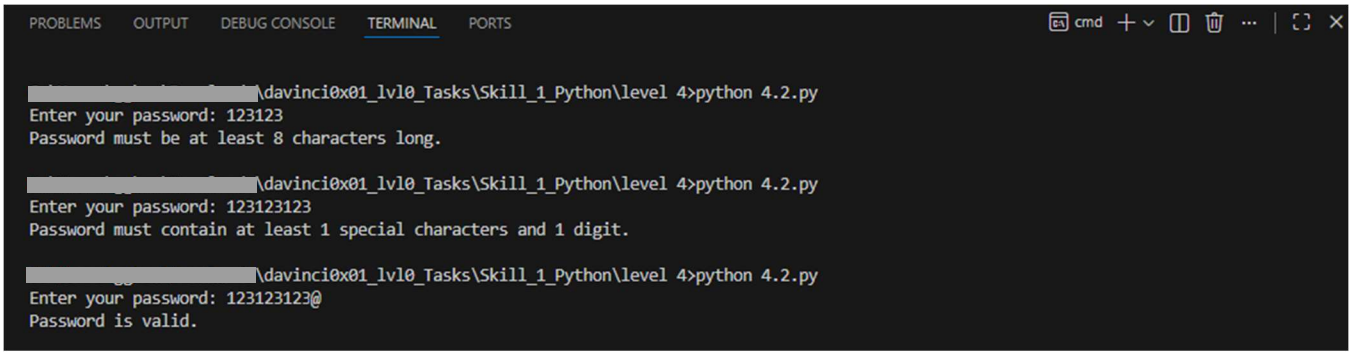
look at 1.5.py at level 1

2. Check if a password is strong > 4.2.py

- Create variable for special character

- Use if to check password length and use any() to the other conditions

Screenshot:



3. Generate a random password of 12 characters > 4.3.py

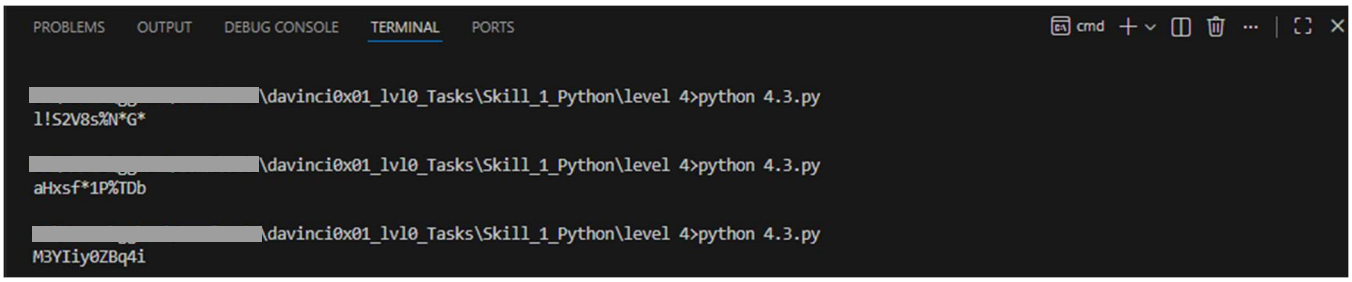
- Import random

- Create variable with allowed password characters and final result

- Type while loop until the password length = 12

- Use random.choice() function to add the choise to password variable

Screenshot:



4. Create an MD5 hash of a user-entered string using hashlib > 4.4.py

- Import hashlib

- Create user input

- Print it converted with hashlib.md5(user_input.encode).hexdigest()

- User input must be encoded and use hexdigest() to return input as string

Screenshot:

5. Check if an IP address is valid > 4.5.py

- First get the IP address from the user

- Split the IP address using " ." to get all parts

- Check if the IP contains exactly 4 parts

- Use for loop to check each part It must be a number and must be between 0–255

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.5.py
Enter an IP address: 192.168.1.1
Valid IP address

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.5.py
Enter an IP address: 256.10.1.5
Invalid IP address

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.5.py
Enter an IP address: 192.168.1
Invalid IP address

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.5.py
Enter an IP address: abc.def.1.1
Invalid IP address
```

6. Generate random MAC addresses > 4.6.py

- Import random

- Create an empty list to store MAC address parts

- Use a for loop to generate 6 random numbers between 0 and 255

- Convert each number to hexadecimal using 02x format

- Join all parts using ":" to create the MAC address

Screenshot:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                    cmd

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.6.py
Random MAC address: 48:da:02:7a:7a:52

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.6.py
Random MAC address: eb:2f:a1:bb:37:84
```

7. XOR encryption function > 4.7.py

- XOR encryption is a simple encryption method based on the XOR (^) operation

- The function takes two inputs: the text to be encrypted and a key

- A for loop is used to loop through each character in the text

- Each character is converted to its ASCII value using ord()

- The XOR operator (^) is applied between the text character and the key character

- The key is repeated using the modulo operator % if it is shorter than the text.

- The result of the XOR operation is converted back to a character using chr()

- The same function is used for encryption and decryption because XOR is reversible

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.7.py
Enter text: davinci
Enter key: secret
Encrypted: ↕♦§d↕→
Decrypted: davinci
```

8. Generate a random device ID using uuid > 4.8.py

- Import uuid and Use uuid.uuid4() to generate a random unique identifier

Screenshot :

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.8.py
Generated Device ID: b43561c9-c370-40dc-8be0-386b5c4d76f4

[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.8.py
Generated Device ID: 8ba23b7c-6996-49a0-a82e-7daa6b59de9c
```

9. Resolve a hostname to an IP address > 4.9.py

- Import socket

- Ask the user to enter a hostname

- Use socket.gethostbyname(hostname) to resolve the hostname into an IP address

- If the hostname cannot be resolved the exception and display an error message

- The program prints the IP address if successful

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.9.py
Enter hostname: google.com
The IP address of google.com is 172.217.18.238
```

10. Extract all vowels from a string > 4.10.py

- Create function with parameter called phrase in it 3 variables for valid phrase and extracted vowels and the vowels

- Create for loop to check for every character in phrase

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_1_Python\level 4>python 4.10.py
Enter a phrase: hello world
Vowels in the phrase: eoo
Phrase without vowels: hll wrld
```